

MAXIME BRONNY

19009314

OUTILS LIBRES POUR LE DÉVELOPPEMENT LOGICIEL

- PROJET ACADÉMIQUE -

PRÉDICTION DU RISQUE DE CRÉDIT BANCAIRE (PIPELINE ML PYTHON/SCIKIT-LEARN, API FLASK, FRONT VUE 3/VITE, DÉPLOIEMENT DOCKER COMPOSE + POSTGRESQL)

PRÉSENTATION

Prédiction du Risque de Crédit Bancaire est un projet réalisé dans le cadre de l'UE Outils libres pour le développement logiciel. L'objectif est de mettre en place un système complet de prédiction du risque de défaut de paiement, de l'entraînement des modèles jusqu'à une interface web utilisable.

Le jeu de données public `credit_risk_dataset.csv` (Kaggle, ~32 500 observations, 11 variables explicatives) est découpé de façon stratifiée (80/20), puis deux modèles sont entraînés (régression logistique et arbre de décision) via un pipeline orchestré par un Makefile. Les prédictions sont exposées par une API REST Flask (validation Marshmallow, persistance PostgreSQL avec SQLAlchemy) et consommées par une interface Vue3 (Tailwind, Vite). L'ensemble est conteneurisé avec Docker Compose (PostgreSQL, API Unicorn, front-end). Le dépôt contient les scripts du pipeline ML, les tests pytest (19 tests), les Dockerfiles et la documentation du rapport.

[HTTPS://GITHUB.COM/CRYZMX](https://github.com/cryzmx)

Sommaire

Liste des sigles et glossaire	5
1 Introduction	8
1.1 Contexte et problématique	8
1.2 Objectifs du projet	8
1.3 Périmètre et contraintes	9
2 Architecture du projet	10
2.1 Vue d'ensemble	10
2.2 Structure du repository	11
2.3 Pipeline ML (offline)	12
2.3.1 Split du dataset	12
2.3.2 Entraînement des modèles	13
2.3.3 Génération des graphiques	13
2.4 API REST (online)	13
2.5 Front-end	14
2.6 Conteneurisation	14
3 Données et modèles	16
3.1 Description du dataset	16
3.2 Prétraitement	17
3.3 Modèles	18
3.4 Résultats	19
3.5 Seuils de décision métier	22
4 Outils et justifications	23
4.1 Grille d'analyse	23
4.2 Langages	23
4.2.1 Python 3.11	23
4.2.2 JavaScript (ES2022)	23
4.3 Pipeline ML	23
4.3.1 scikit-learn	24
4.3.2 pandas / NumPy	24
4.3.3 matplotlib / seaborn	24
4.3.4 joblib	24
4.4 API back-end	25

4.4.1	Flask	25
4.4.2	Marshmallow	25
4.4.3	SQLAlchemy	25
4.4.4	Gunicorn	25
4.4.5	Flask-CORS / psycpg2-binary	26
4.5	Front-end	26
4.5.1	Vue 3	26
4.5.2	Vite	26
4.5.3	Tailwind CSS 4	26
4.5.4	Vue Router 4	27
4.6	Infrastructure	27
4.6.1	Docker / Docker Compose	27
4.6.2	PostgreSQL 16	27
4.6.3	Make	27
4.7	Gestion de versions	28
4.7.1	Git	28
4.8	Tests et qualité	28
4.8.1	pytest	28
4.8.2	SQLite in-memory	28
4.8.3	pytest-cov	29
4.8.4	GitHub Actions (intégration continue)	29
4.9	Tableau récapitulatif	29
4.10	Licence du projet	30
5	Environnement et reproductibilité	31
5.1	Installation locale	31
5.2	Déploiement Docker	32
5.3	Exécution du pipeline	33
5.4	Tableau des versions	33
6	Tests	35
6.1	Stratégie de tests	35
6.2	Isolation	35
6.3	Exécution	36
6.4	Intégration continue et couverture	36
7	Limites et améliorations	37
7.1	Limites actuelles	37
7.2	Améliorations possibles	37
8	Conclusion	38

Bibliographie	39
A Arborescence complète du repository	41
B Exemple de requête/réponse API	43
C Captures d'écran de l'interface web	44

Liste des sigles et glossaire

Sigles et acronymes

API	Interface permettant à deux applications de communiquer.
AUC-ROC	Métrique évaluant la capacité d'un modèle à distinguer les classes.
CI	<i>Continuous Integration</i> , exécution automatique des tests à chaque modification du dépôt.
CORS	Mécanisme autorisant un site web à appeler une API hébergée sur une autre origine.
CSV	Format de fichier texte représentant des données tabulaires séparées par des virgules.
DT	<i>Decision Tree</i> , arbre de décision.
JSON	Format d'échange de données textuel utilisé entre le front-end et l'API.
LR	<i>Logistic Regression</i> , régression logistique.
ML	<i>Machine Learning</i> , méthode permettant d'entraîner un modèle à partir de données.
ORM	<i>Object-Relational Mapping</i> , technique faisant correspondre des objets du code à des tables SQL.
REST	Style d'architecture utilisé pour exposer des services web via HTTP.
SPA	<i>Single Page Application</i> , application web monopage.
WSGI	Interface standard entre un serveur HTTP et une application web Python.

Glossaire

Accuracy	Proportion de prédictions correctes.
Arbre de décision	Modèle de classification basé sur une succession de règles de décision.
Back-end	Partie serveur de l'application qui traite les requêtes et exécute la logique métier.
Copyleft	Famille de licences libres (ex. GPL) imposant que les redistributions restent sous la même licence, par opposition aux licences permissives (MIT, BSD, Apache).
Credit scoring	Évaluation du risque de défaut d'un emprunteur.
Dataset	Jeu de données utilisé pour entraîner et tester les modèles.
Docker	Outil de conteneurisation permettant d'exécuter l'application dans un environnement isolé.
Docker Compose	Outil permettant d'orchestrer plusieurs conteneurs.
F1-score	Moyenne harmonique entre precision et recall.
Feature	Variable explicative utilisée par un modèle.
Flask	Framework Python utilisé pour développer l'API.
Front-end	Partie visible de l'application utilisée par l'utilisateur.
Gunicorn	Serveur WSGI utilisé pour exécuter l'API Flask.
joblib	Format utilisé pour sérialiser les modèles entraînés.
Makefile	Fichier permettant d'automatiser les commandes du projet.
Marshmallow	Bibliothèque Python utilisée pour valider les données envoyées à l'API.
Pipeline ML	Suite d'étapes allant du chargement des données à l'entraînement et à l'évaluation des modèles.
PostgreSQL	Base de données relationnelle utilisée pour stocker les prédictions.
Precision	Proportion de prédictions positives réellement correctes.
Recall	Proportion de vrais défauts correctement détectés.
Régression logistique	Modèle statistique utilisé ici pour prédire une probabilité de défaut.
SQLAlchemy	ORM Python utilisé pour communiquer avec la base de données.

Target	Variable cible que le modèle cherche à prédire.
Vite	Outil de développement et de build utilisé pour le front-end.
Vue 3	Framework JavaScript utilisé pour l'interface web.

Chapitre 1

Introduction

1.1 Contexte et problématique

L'octroi de crédit constitue une activité centrale du secteur bancaire, dans laquelle l'établissement prêteur doit évaluer la probabilité qu'un emprunteur ne rembourse pas son prêt. Cette évaluation, appelée *credit scoring*, se formule comme un problème de classification binaire : la variable cible `loan_status` vaut 0 lorsque l'emprunteur honore ses échéances et 1 en cas de défaut de paiement. Une mauvaise estimation du risque entraîne soit des pertes financières (faux négatifs), soit un refus injustifié de demandes solvables (faux positifs).

Le présent projet s'appuie sur le dataset public `credit_risk_dataset.csv`, disponible sur Kaggle [10]. Ce jeu de données contient 32 581 observations décrites par 11 variables explicatives (âge, revenu, montant du prêt, taux d'intérêt, grade, historique de crédit, etc.) et une variable cible binaire. La distribution des classes est déséquilibrée : 78,18 % des observations correspondent à une absence de défaut (classe 0) contre 21,82 % de défauts (classe 1). Ce déséquilibre, fréquent en pratique, influence directement le choix des métriques d'évaluation et les seuils de décision.

1.2 Objectifs du projet

Ce projet vise à construire un système complet de prédiction du risque de crédit, depuis l'entraînement des modèles jusqu'à leur mise à disposition via une interface web. Les objectifs sont les suivants : premièrement, mettre en place un pipeline d'apprentissage automatique reproductible, orchestré par un Makefile, qui enchaîne la séparation des données, l'entraînement de deux modèles (régression logistique et arbre de décision) et la génération de graphiques d'évaluation. Deuxièmement, exposer les modèles entraînés à travers une API REST développée avec Flask [17] et servie par Gunicorn en production. Troisièmement, fournir une interface web construite avec Vue 3 [25] et Tailwind CSS [24], permettant à un utilisateur de soumettre un profil d'emprunteur et de consulter les résultats de prédiction. Quatrièmement, conteneuriser l'ensemble de l'application (base de données PostgreSQL [19], API, front-end) via Docker Compose [4] afin de garantir la portabilité du déploiement. Enfin, valider le bon fonctionnement du système par une suite de 25 tests automatisés couvrant la validation des entrées, le prétraitement, la logique de prédiction et les endpoints de l'API.

1.3 Périmètre et contraintes

Le projet se concentre sur la démonstration d'un pipeline ML de bout en bout utilisant exclusivement des outils libres. Plusieurs simplifications ont été retenues volontairement. L'API ne comporte pas de mécanisme d'authentification : elle est accessible sans restriction, ce qui est acceptable dans un contexte académique mais inadapté à un déploiement en production. Les tests sont exécutables localement via `$ make test` et rejoués automatiquement par une intégration continue GitHub Actions à chaque modification du dépôt. Seuls deux algorithmes de classification sont implémentés - la régression logistique et l'arbre de décision - afin de comparer un modèle linéaire à un modèle non linéaire sans recourir à des techniques d'ensemble. Le dataset est statique : aucun mécanisme de réentraînement en ligne n'est prévu, les modèles étant sérialisés une fois puis chargés au démarrage de l'API.

Chapitre 2

Architecture du projet

2.1 Vue d'ensemble

Le système se décompose en deux parties distinctes, illustrées en Figure 2.1. La première est un pipeline ML *offline*, exécuté en ligne de commande via **Make**, qui prend en entrée le dataset brut, produit les modèles sérialisés au format **joblib** et génère les métriques et graphiques d'évaluation. La seconde est une application web *online*, déployée via Docker Compose, qui suit une architecture trois tiers classique : un front-end Vue 3 communique en JSON/REST avec une API Flask, laquelle persiste les prédictions dans une base PostgreSQL. Le lien entre les deux parties est matérialisé par les fichiers **.joblib** : produits par le pipeline offline, ils sont chargés au démarrage de l'API pour effectuer les prédictions en temps réel.

Pipeline ML hors ligne - `make all`

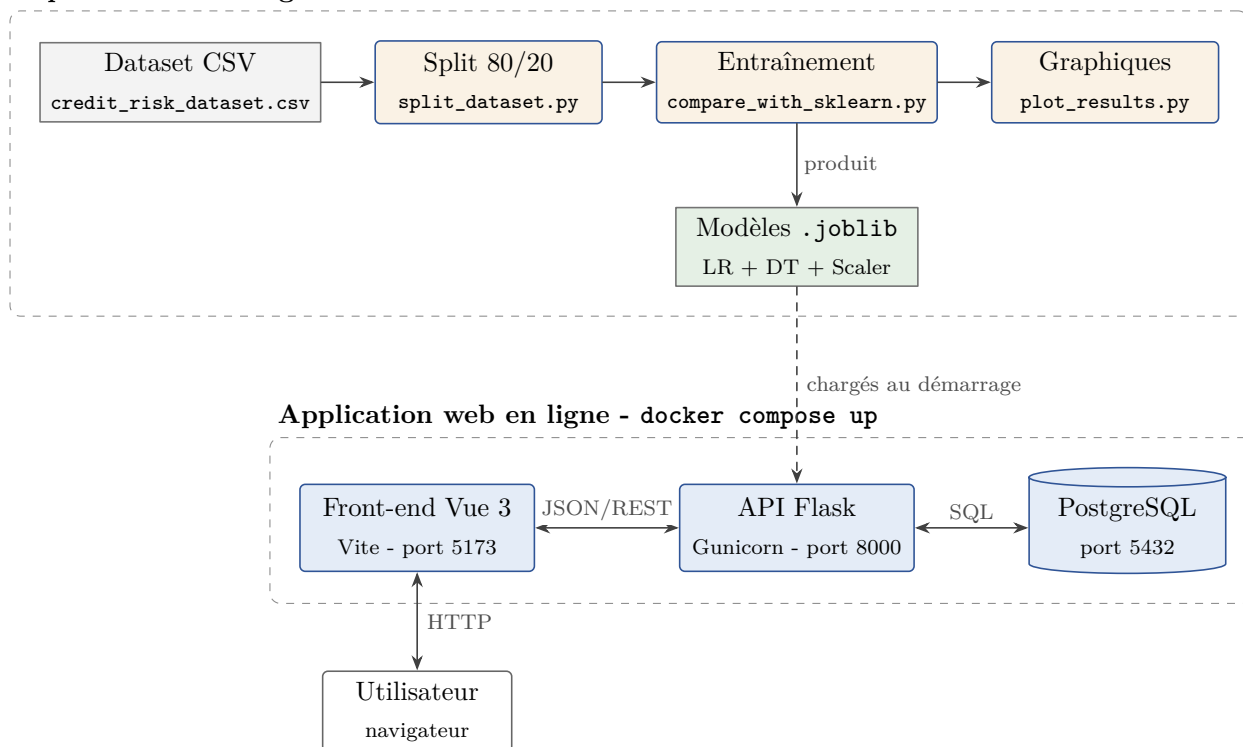


Figure 2.1 - Architecture générale du projet : pipeline ML hors ligne et application web.

2.2 Structure du repository

Le dépôt est organisé selon une séparation claire entre le back-end Python et le front-end JavaScript. Le répertoire **back-end/app/** contient les huit modules de l'API Flask (point d'entrée, routes, prédiction, prétraitement, modèles ORM, schémas de validation, configuration, accès base de données). Le répertoire **back-end/scripts/** regroupe les six scripts du pipeline ML (split, entraînement, visualisation, exploration, détection d'outliers, analyse des valeurs manquantes). Les modèles sérialisés sont stockés dans **back-end/models/**, les résultats dans **back-end/results/**. Le front-end, dans **front-end/src/**, suit l'organisation standard d'un projet Vue 3 avec des vues, des composants, un routeur et un client API. Les fichiers d'orchestration se trouvent à la racine (**Makefile**, **docker-compose.yml**, **Dockerfile** du pipeline), tandis que les Dockerfiles de l'API et du front-end résident dans leurs répertoires respectifs (**back-end/Dockerfile.api**, **front-end/Dockerfile**).

```
PROGRAMME/
+-- back-end/
|   +-- app/          # API Flask
|   +-- scripts/      # Pipeline ML
|   +-- tests/        # Tests pytest (25 tests)
|   +-- data/         # Dataset brut et traité
|   +-- models/       # Modèles .joblib
|   +-- results/      # Métriques et graphiques
+-- front-end/
|   +-- src/          # Vue 3 (views, components)
+-- docs/             # Documentation et rapport
+-- Makefile          # Orchestration du pipeline
+-- docker-compose.yml
+-- Dockerfile
```

2.3 Pipeline ML (offline)

Le pipeline ML est orchestré par un Makefile [18] qui déclare les dépendances entre étapes : `$ make all` enchaîne séquentiellement le split du dataset, l'entraînement des modèles et la génération des graphiques. Le flux complet est représenté en Figure 2.2. Chaque étape correspond à un script Python indépendant, ce qui facilite la ré-exécution partielle : par exemple, relancer uniquement `$ make plots` sans réentraîner les modèles. Le Makefile définit également une cible `$ make train-force` qui force le réentraînement en ignorant les modèles .joblib déjà existants, via la variable d'environnement `FORCE_RETRAIN=1`.

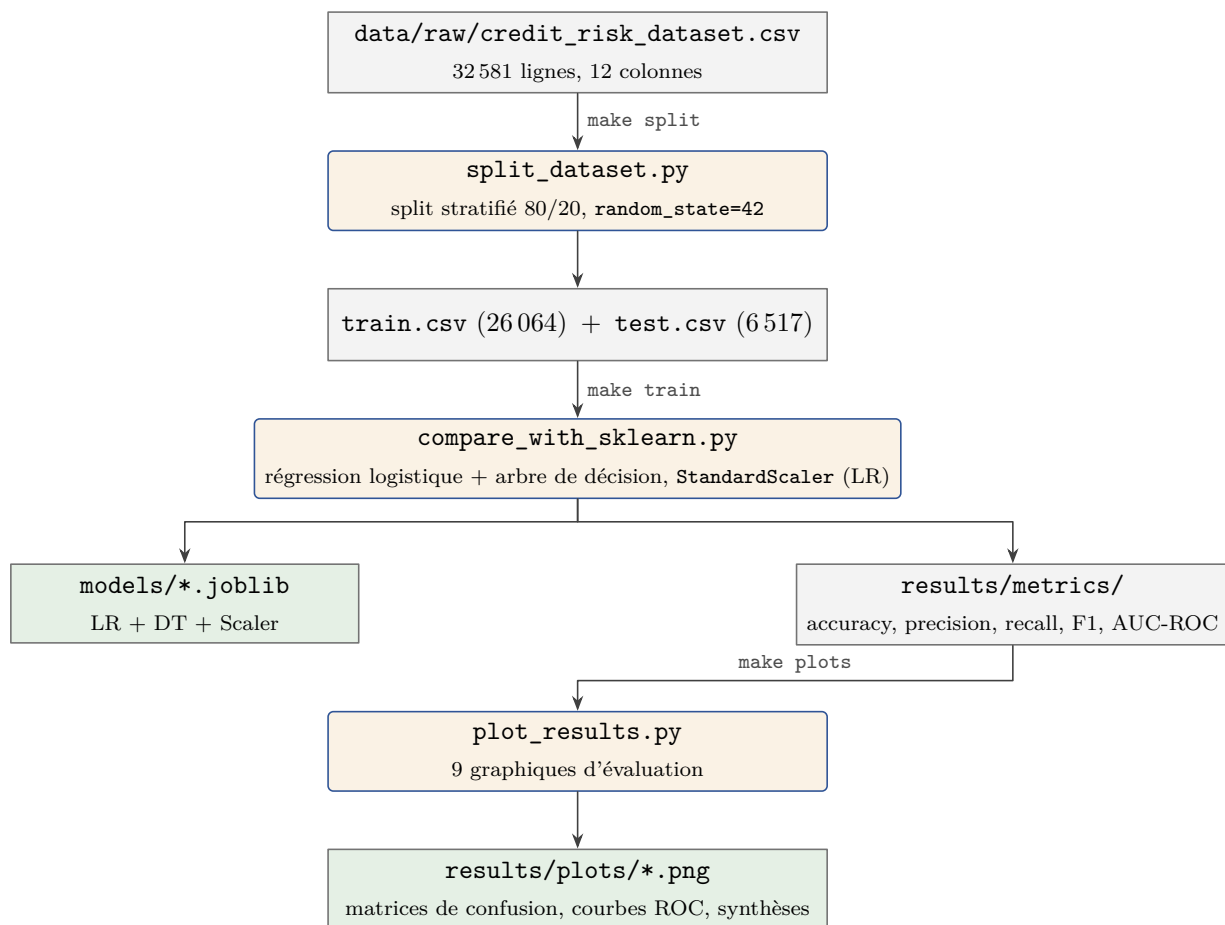


Figure 2.2 - Chaîne de traitement du pipeline d'apprentissage automatique (`$ make all`).

2.3.1 Split du dataset

Le script `split_dataset.py` charge le fichier brut `credit_risk_dataset.csv` (32 581 lignes) et réalise un split stratifié 80/20 à l'aide de la fonction `train_test_split` de scikit-learn [15], avec `random_state=42` pour assurer la reproductibilité. La stratification garantit que la proportion de la classe minoritaire (21,82 %) est identique dans les ensembles d'entraînement (26 064 lignes) et de test (6 517 lignes), comme le confirme le Tableau 3.2. Les fichiers résultants sont écrits dans `back-end/data/processed/`.

2.3.2 Entraînement des modèles

Le script `compare_with_sklearn.py` entraîne deux modèles scikit-learn sur l'ensemble d'entraînement. Les variables catégorielles sont encodées numériquement via des mappings déterministes (identiques à ceux de l'API), puis un `StandardScaler` est ajusté et appliqué aux features pour la régression logistique. L'arbre de décision, invariant aux transformations d'échelle, utilise les données non normalisées. Les trois artefacts produits (modèle LR, modèle DT, scaler) sont sérialisés au format `.joblib` dans `back-end/models/`. Les métriques d'évaluation (accuracy, precision, recall, F1-score, AUC-ROC) et les matrices de confusion sont calculées sur les deux ensembles (train et test) et sauvegardées dans `back-end/results/metrics/`.

2.3.3 Génération des graphiques

Le script `plot_results.py` lit les fichiers de métriques et les données ROC produits à l'étape précédente, puis génère neuf graphiques au format PNG à l'aide de matplotlib [9] : les matrices de confusion pour chaque modèle, les courbes ROC comparatives, les comparaisons train/test des métriques, la courbe de coût de la régression logistique, ainsi que trois figures de synthèse. Ces graphiques sont utilisés directement dans le présent rapport (voir Figures 3.1 et 3.2).

2.4 API REST (online)

L'API REST, définie dans `back-end/app/routes.py`, expose quatre endpoints résumés dans le Tableau 2.1. L'endpoint principal `POST /predict` suit le flux détaillé en Figure 2.3 : le payload JSON est d'abord validé par un schéma Marshmallow [12] qui vérifie les types, les bornes numériques et les valeurs catégorielles autorisées. En cas d'erreur, une réponse 400 est retournée avec le détail des champs invalides. Si la validation réussit, les données sont encodées numériquement par le module `preprocess.py`, puis transmises aux fonctions `predict_lr` et/ou `predict_dt` selon le paramètre `model_choice` (valeurs possibles : `lr`, `dt` ou `both`). Chaque prédiction retourne une probabilité de défaut, un niveau de risque et une décision de crédit. L'ensemble des résultats est persisté dans la base de données via SQLAlchemy [1], et une réponse 201 contenant l'identifiant de l'enregistrement est renvoyée au client.

Tableau 2.1 - Endpoints de l'API REST

Méthode	Path	Description	Codes
GET	<code>/health</code>	Vérification de l'état de l'API	200
POST	<code>/predict</code>	Prédiction de risque (LR et/ou DT)	201, 400
GET	<code>/predictions</code>	Historique des prédictions (paginé)	200
GET	<code>/predictions/<id></code>	Détail d'une prédiction	200, 404

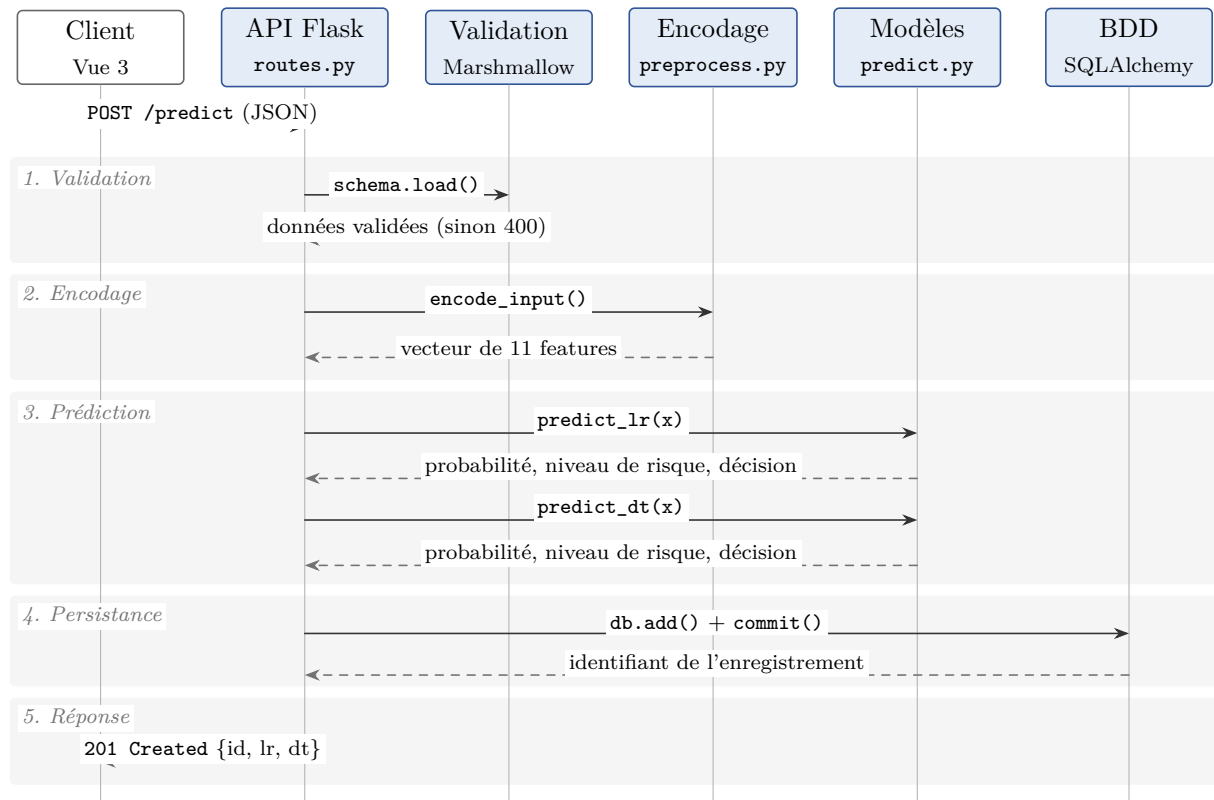


Figure 2.3 - Flux d'une requête de prédiction via l'API REST (POST /predict).

2.5 Front-end

L'interface utilisateur est une application monopage (SPA) construite avec Vue 3 [25] en mode Composition API et stylisée avec Tailwind CSS [24]. Elle comporte trois vues, gérées par Vue Router [22]. La vue principale (**PredictView**) affiche un formulaire de 11 champs correspondant aux features du modèle, avec un champ supplémentaire pour le choix du modèle. Le ratio prêt/revenu est calculé automatiquement côté client via un **watch** réactif. À la soumission, le composant appelle l'API via le module **client.js** et affiche les résultats dans deux cartes **ResultCard** (une par modèle), chacune présentant la probabilité de défaut sous forme de barre de progression colorée, le niveau de risque et la décision de crédit. La vue **HistoryView** récupère la liste paginée des prédictions et les affiche dans un tableau triable. Enfin, **DetailView** présente le détail complet d'une prédiction passée, incluant les données saisies et les résultats des deux modèles.

2.6 Conteneurisation

Le déploiement de l'application repose sur Docker Compose [4], qui orchestre trois conteneurs décrits dans le Tableau 2.2. Le service **db** utilise l'image officielle **postgres:16-alpine** et persiste ses données dans un volume nommé **pgdata**. Le service **api** est construit à partir de **Dockerfile.api** (base **python:3.11-slim**) : il installe les dépendances **pip**, copie le code applicatif et les modèles **.joblib**, puis démarre Gunicorn [7] sur le port 8000.

Le service `web` est construit à partir de `front-end/Dockerfile` (base `node:20-alpine`) et lance le serveur de développement Vite [26] sur le port 5173. Les variables d'environnement `DATABASE_URL` et `VITE_API_URL` sont injectées par le fichier Compose pour connecter les services entre eux. L'ordre de démarrage est garanti par les directives `depends_on` associées à des `healthcheck` : l'API n'est lancée que lorsque PostgreSQL accepte réellement les connexions (`pg_isready`), et le front-end que lorsque l'endpoint `/health` de l'API répond. La cible `$ make docker-up` enchaîne en outre l'entraînement des modèles puis le déploiement, évitant le cas d'une API démarrée sans fichiers `.joblib`.

Tableau 2.2 - Services Docker Compose

Service	Image	Port	Rôle
db	postgres:16-alpine	5432	Base de données PostgreSQL (persistance des prédictions)
api	python:3.11-slim	8000	API Flask + Gunicorn (prédiction et CRUD)
web	node:20-alpine	5173	Front-end Vue 3 + Vite (serveur de développement)

Chapitre 3

Données et modèles

3.1 Description du dataset

Le dataset utilisé est `credit_risk_dataset.csv` [10], un jeu de données tabulaire composé de 32 581 observations et 12 colonnes.

Le jeu de données ne concerne pas les prêts immobiliers (crédit maison sur 15 ou 25 ans), mais des prêts à la consommation et à court terme. La variable `loan_intent` indique l'usage du prêt : personnel (*personal*), études (*education*), santé (*medical*), création d'entreprise (*venture*), travaux d'amélioration du logement (*home improvement*) ou consolidation de dettes (*debt consolidation*). Il s'agit donc de financements dont les montants et les durées sont ceux du crédit conso ou du prêt personnel (quelques milliers à quelques dizaines de milliers de dollars, remboursement sur quelques années). L'application de prédiction et les modèles entraînés sont adaptés à ce type de demande : un utilisateur qui saisit un profil dans l'interface simule une demande de crédit à la consommation (ou prêt personnel), et non une demande de crédit immobilier.

Les 11 variables explicatives, détaillées dans le Tableau 3.1, se répartissent en sept variables numériques (âge, revenu, années d'emploi, montant du prêt, taux d'intérêt, ratio prêt/revenu, ancienneté de crédit) et quatre variables catégorielles (statut de propriété, objet du prêt, grade du prêt, antécédent de défaut). La variable cible `loan_status` est binaire. Le script `explore_data.py` permet d'obtenir les statistiques descriptives, les distributions et la matrice de corrélation du dataset. Le script `analyze_missing_values.py` confirme l'absence de valeurs manquantes après chargement, et `detect_outliers.py` identifie les valeurs aberrantes par les méthodes IQR et Z-score sur les variables numériques.

Tableau 3.1 - Features du dataset `credit_risk_dataset.csv`

Feature	Type	Description	Valeurs
<code>person_age</code>	Num.	Âge du demandeur	18-100
<code>person_income</code>	Num.	Revenu annuel (\$)	0-10 M
<code>person_home_ownership</code>	Cat.	Statut de propriété	RENT, OWN, MORTGAGE, OTHER
<code>person_emp_length</code>	Num.	Années d'emploi	0-100
<code>loan_intent</code>	Cat.	Objet du prêt	PERSONAL, EDUCATION, ...
<code>loan_grade</code>	Cat.	Grade du prêt	A-G
<code>loan_amnt</code>	Num.	Montant du prêt (\$)	0 - 100 000
<code>loan_int_rate</code>	Num.	Taux d'intérêt (%)	0-30
<code>loan_percent_income</code>	Num.	Ratio prêt/revenu	0 - 1
<code>cb_person_default_on_file</code>	Cat.	Antécédent de défaut	N, Y
<code>cb_person_cred_hist_length</code>	Num.	Historique crédit (ans)	0-50
<code>loan_status</code> (cible)	Bin.	Défaut de paiement	0 (non), 1 (oui)

3.2 Prétraitement

Le prétraitement est implémenté dans le module `preprocess.py` et répliqué à l'identique dans le script d'entraînement pour garantir la cohérence entre les phases offline et online. Les quatre variables catégorielles sont encodées via des dictionnaires de correspondance déterministes : `person_home_ownership` est transformé en entier (RENT→0, OWN→1, MORTGAGE→2, OTHER→3), `loan_intent` en six valeurs ordinales, `loan_grade` en entier de 1 (A) à 7 (G), et `cb_person_default_on_file` en binaire (N→0, Y→1). L'ordre des 11 features dans le vecteur résultant est fixé par la constante `FEATURE_ORDER`, partagée entre l'API et le script d'entraînement.

Le dataset contient 3 116 valeurs manquantes sur `loan_int_rate` et 895 sur `person_emp_length`. Elles sont imputées par la moyenne, avec une précaution essentielle contre la fuite de données (*data leakage*) : les moyennes d'imputation sont calculées sur l'ensemble d'entraînement uniquement, puis appliquées à l'identique au train et au test (fonctions `compute_imputation_value` et `apply_imputation` du script d'entraînement). Une première version du script imputait chaque ensemble avec ses propres moyennes, laissant des informations du test influencer son propre prétraitement ; cette erreur méthodologique a été corrigée et trois tests automatisés (`test_imputation.py`) vérifient désormais que les statistiques d'imputation proviennent

exclusivement du train. La correction n'a pas modifié les métriques mesurées - les moyennes des deux ensembles sont très proches sur des échantillons de cette taille - mais elle garantit la validité méthodologique de l'évaluation.

Pour la régression logistique, un `StandardScaler` de scikit-learn [15] est ajusté sur les données d'entraînement puis appliqué aux données de test et aux requêtes API. Ce scaler centre et réduit chaque feature (moyenne 0, écart-type 1), ce qui est nécessaire pour la convergence de l'algorithme L-BFGS. L'arbre de décision, dont les décisions reposent sur des seuils bruts, n'utilise pas cette normalisation. Le split stratifié, détaillé dans le Tableau 3.2, préserve la distribution de la classe cible dans les deux ensembles.

Tableau 3.2 - Statistiques du split stratifié

Ensemble	Échantillons	Classe 1 (%)
Dataset complet	32 581	21,82 %
Train set	26 064	21,82 %
Test set	6 517	21,82 %

3.3 Modèles

Deux modèles de classification sont entraînés et comparés, avec les hyperparamètres détaillés dans le Tableau 3.4. La régression logistique utilise l'algorithme d'optimisation L-BFGS (`solver='lbfgs'`) avec un maximum de 1 000 itérations pour garantir la convergence. Ce modèle linéaire produit directement une probabilité de défaut via la fonction sigmoïde, ce qui facilite l'interprétation des seuils de décision.

L'arbre de décision est configuré avec une profondeur maximale de 7, un minimum de 20 échantillons pour effectuer une division et un minimum de 10 échantillons par feuille. Ces contraintes de régularisation visent à limiter le surapprentissage tout en conservant une capacité discriminante suffisante. Le critère de Gini est utilisé pour mesurer l'impureté des nœuds. L'arbre résultant comporte 111 nœuds. Les deux modèles utilisent `random_state=42` pour la reproductibilité.

Le choix de la profondeur maximale est justifié par une validation croisée stratifiée à 5 plis (`StratifiedKfold`), menée sur l'ensemble d'entraînement uniquement, qui compare les profondeurs 3, 5, 7, 9 et 11 à hyperparamètres par ailleurs identiques (Tableau 3.3). Le F1-score culmine aux profondeurs 7 (0,810) et 9 (0,811), deux valeurs statistiquement indiscernables (l'écart est inférieur à un écart-type), tandis que l'AUC-ROC est maximale à la profondeur 7 (0,905) puis décroît, signe d'un début de surapprentissage. À performances équivalentes, la profondeur 7 est retenue par principe de parcimonie : l'arbre reste plus compact et plus lisible.

Tableau 3.3 - Validation croisée stratifiée 5-fold pour le choix de `max_depth` (train set unique-ment)

<code>max_depth</code>	F1 (moy. $\pm$ é.-t.)	Recall	AUC-ROC
3	0,679 $\pm$ 0,023	0,548	0,854
5	0,754 $\pm$ 0,009	0,611	0,895
7	0,810 $\pm$ 0,011	0,697	0,905
9	0,811 $\pm$ 0,014	0,705	0,904
11	0,806 $\pm$ 0,014	0,707	0,900

Tableau 3.4 - Hyperparamètres des modèles

Modèle	Paramètre	Valeur	Rôle
Rég. Logistique	<code>solver</code>	<code>lbfgs</code>	Algorithme d'optimisation L-BFGS
	<code>max_iter</code>	1000	Nombre max d'itérations
	<code>random_state</code>	42	Reproductibilité
Arbre de Décision	<code>max_depth</code>	7	Profondeur maximale
	<code>min_samples_split</code>	20	Min. échantillons pour diviser
	<code>min_samples_leaf</code>	10	Min. échantillons par feuille
	<code>criterion</code>	<code>gini</code>	Critère d'impureté
	<code>random_state</code>	42	Reproductibilité

3.4 Résultats

Les résultats comparatifs des deux modèles sont présentés dans le Tableau 3.6. L'arbre de décision surpasse la régression logistique sur l'ensemble des métriques : son accuracy atteint 92,73 % contre 84,92 % pour la LR, et son AUC-ROC de 0,9063 témoigne d'une meilleure capacité de discrimination globale (voir Figure 3.2). L'écart est particulièrement marqué sur le recall (70,39 % contre 48,17 %), ce qui signifie que l'arbre de décision détecte davantage de défauts réels, un critère souvent prioritaire en credit scoring.

Le Tableau 3.7 et la Figure 3.1 détaillent les matrices de confusion. La régression logistique génère un nombre élevé de faux négatifs (737 défauts non détectés sur 1 422) tandis que l'arbre de décision en produit sensiblement moins (421). En contrepartie, l'arbre maintient un taux de faux positifs très faible (53 contre 246 pour la LR), ce qui se traduit par une précision de 94,97 %.

La faible différence entre les métriques train et test pour les deux modèles (moins de 1 point de pourcentage sur l'accuracy) indique l'absence de surapprentissage significatif. Ce résultat est cohérent avec la régularisation appliquée à l'arbre de décision (profondeur limitée à 7) et la simplicité inhérente du modèle logistique.

Le déséquilibre des classes (78/22) rend l'accuracy insuffisante à elle seule : un classifieur naïf prédisant systématiquement « pas de défaut » atteindrait déjà 78 % d'accuracy sans détecter le moindre défaut. L'analyse privilégie donc le recall, le F1-score et l'AUC-ROC. Dans le contexte du crédit, les deux types d'erreurs n'ont d'ailleurs pas le même coût : un faux négatif (défaut non détecté) entraîne une perte financière directe pour le prêteur, tandis qu'un faux positif (client solvable refusé) représente un manque à gagner. Pour mesurer l'effet d'un traitement explicite du déséquilibre, des variantes des deux modèles ont été entraînées avec `class_weight="balanced"`, qui repondère la classe minoritaire (Tableau 3.5). La régression logistique pondérée fait passer le recall de 0,482 à 0,771 (au prix d'une precision réduite à 0,507) et améliore légèrement le F1-score ; l'arbre de décision pondéré, en revanche, n'apporte rien (F1 de 0,802 contre 0,809). L'arbre standard, meilleur modèle global, est donc conservé comme modèle de référence ; la variante pondérée de la régression logistique serait en revanche le bon choix si la LR était le modèle servi et que la détection des défauts primait, les seuils métier du Tableau 3.8 offrant un second levier d'ajustement.

Tableau 3.5 - Effet de `class_weight="balanced"` sur le test set (variantes d'étude, non déployées)

Modèle	Accuracy	Precision	Recall	F1	AUC-ROC
LR standard	0,849	0,736	0,482	0,582	0,852
LR balanced	0,787	0,507	0,771	0,612	0,853
DT standard	0,927	0,950	0,704	0,809	0,906
DT balanced	0,922	0,904	0,721	0,802	0,901

Tableau 3.6 - Métriques comparatives : Régression Logistique vs Arbre de Décision

Métrique	Régression Logistique		Arbre de Décision	
	Train	Test	Train	Test
Accuracy	0,8473	0,8492	0,9307	0,9273
Precision	0,7329	0,7358	0,9650	0,9497
Recall	0,4719	0,4817	0,7079	0,7039
F1-Score	0,5741	0,5822	0,8167	0,8086
AUC-ROC	-	0,8521	-	0,9063

Tableau 3.7 - Matrices de confusion sur le test set

Modèle	TN	FP	FN	TP
Régression Logistique	4 849	246	737	685
Arbre de Décision	5 042	53	421	1 001

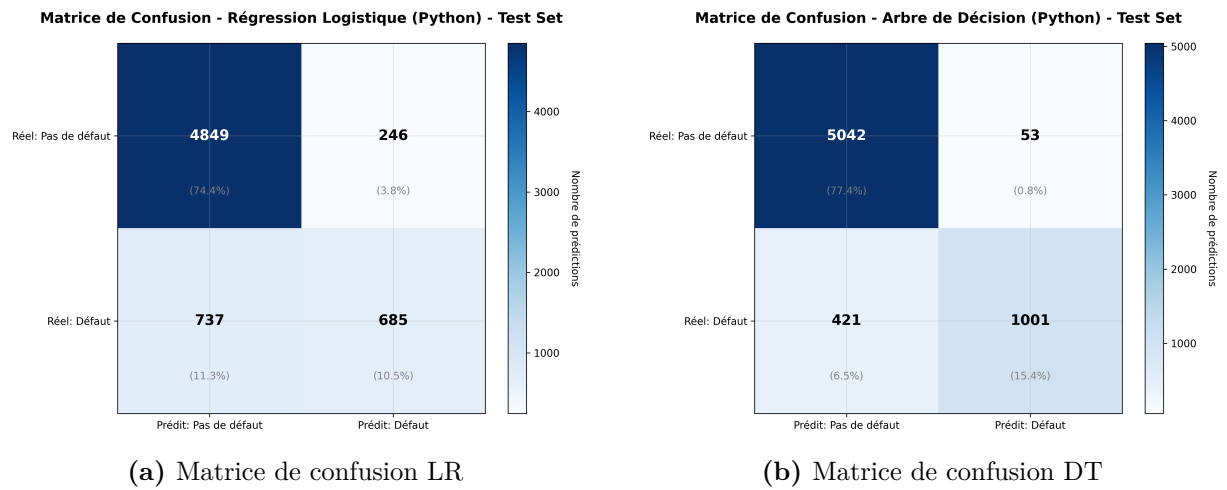


Figure 3.1 - Matrices de confusion sur le test set.

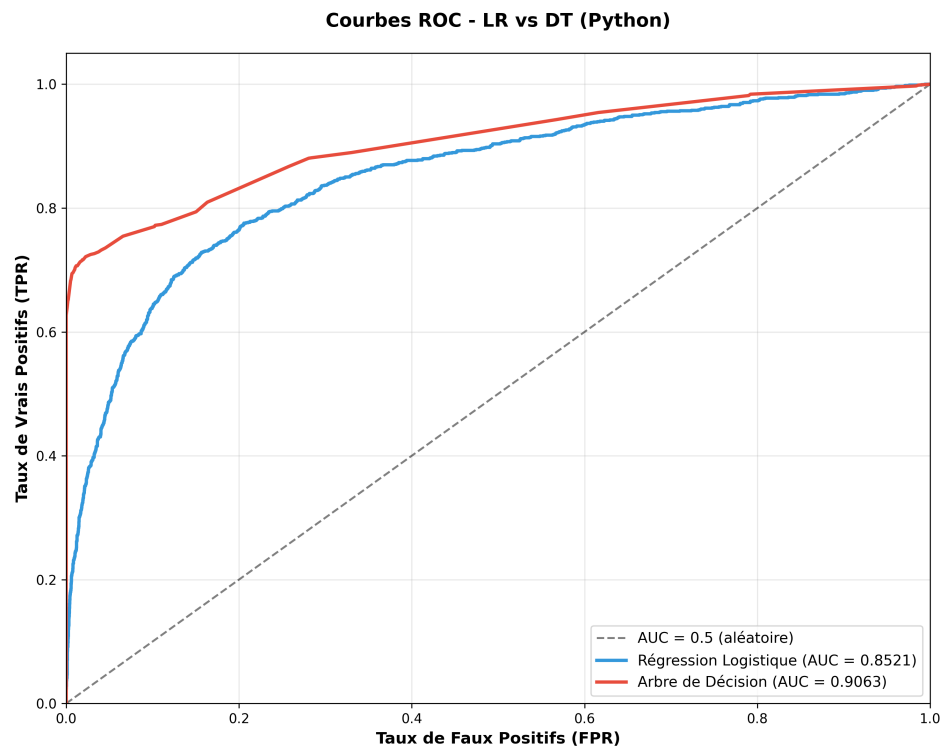


Figure 3.2 - Courbes ROC : comparaison LR vs DT.

3.5 Seuils de décision métier

La probabilité brute produite par chaque modèle est transformée en décision de crédit par la fonction `_classify()` du module `predict.py`, selon les trois seuils définis dans le Tableau 3.8. Une probabilité de défaut inférieure à 30 % est considérée comme sûre et entraîne l'acceptation totale du prêt. Entre 30 % et 60 %, le profil est jugé moyennement risqué et le prêt est accepté partiellement (avec des conditions renforcées, par exemple). Au-delà de 60 %, le prêt est refusé. Ces seuils constituent une politique conservatrice adaptable : dans un contexte réel, ils seraient calibrés en fonction de l'appétence au risque de l'établissement prêteur et du coût relatif des faux positifs et faux négatifs.

Tableau 3.8 - Seuils de classification métier

Probabilité	Niveau de risque	Décision
< 0,30	Safe	Accepté totalement
0,30 - 0,60	Moyennement à risque	Accepté partiellement
> 0,60	À risque	Refusé

Chapitre 4

Outils et justifications

4.1 Grille d'analyse

Chaque outil est évalué selon une grille standardisée : besoin couvert, alternatives envisagées, critères de choix, version, installation, configuration, limites et licence. Cette approche permet une comparaison objective et justifie chaque décision technique.

4.2 Langages

4.2.1 Python 3.11

Python est utilisé pour l'ensemble du back-end : scripts d'analyse, pipeline ML et API REST. Les alternatives considérées étaient R (écosystème statistique riche mais moins adapté au développement web), Julia (performant mais écosystème moins mature) et Java (verbeux pour du prototypage ML). Python a été retenu pour son écosystème ML dominant (scikit-learn, pandas, NumPy), sa syntaxe concise et la disponibilité de frameworks web légers comme Flask. La version 3.11 est imposée par l'image Docker `python:3.11-slim`. La principale limite de Python reste le GIL (*Global Interpreter Lock*), qui empêche le parallélisme CPU natif, mais ce n'est pas contraignant ici puisque les prédictions sont séquentielles et rapides. Licence : PSF License (libre).

4.2.2 JavaScript (ES2022)

JavaScript est le langage du front-end, exécuté dans le navigateur via Vue 3 et Vite. Les alternatives envisagées étaient TypeScript (typage statique, mais surcoût de configuration pour un projet de cette taille), Dart via Flutter Web (écosystème encore restreint côté web) et Streamlit (Python, mais limité en personnalisation d'interface). JavaScript ES2022 a été retenu pour sa compatibilité native avec l'écosystème Vue/Vite et l'absence de phase de compilation. La version est déterminée par Node 20 (image Docker `node:20-alpine`). La principale limite est l'absence de typage statique, qui pourrait compliquer la maintenance sur un projet plus large.

4.3 Pipeline ML

4.3.1 scikit-learn

Scikit-learn [15] fournit l'ensemble des fonctionnalités ML du projet : entraînement des modèles (`LogisticRegression`, `DecisionTreeClassifier`), normalisation (`StandardScaler`), split stratifié (`train_test_split`) et métriques d'évaluation (`accuracy_score`, `f1_score`, `roc_auc_score`, etc.). Les alternatives considérées étaient TensorFlow et PyTorch (surdimensionnés pour des modèles classiques sans deep learning), XGBoost et LightGBM (pertinents mais introduisant une dépendance supplémentaire pour un gain incertain sur ce dataset). Scikit-learn a été retenu pour la cohérence de son API, sa documentation exhaustive et son adéquation avec des modèles interprétables. Version $\geq 1.2.0$, licence BSD-3-Clause. Limite : pas de support GPU ni de modèles profonds.

4.3.2 pandas / NumPy

Pandas [13] et NumPy [8] assurent respectivement la manipulation des données tabulaires (chargement CSV, filtrage, agrégation) et les opérations numériques sur les tableaux multidimensionnels. Les alternatives étaient Polars (plus rapide sur les gros volumes, mais interopérabilité moindre avec scikit-learn) et Dask (calcul distribué, non nécessaire ici avec 32 581 lignes). Ces deux bibliothèques constituent le socle standard de l'écosystème Python pour la data science. Versions $\geq 1.5.0$ et $\geq 1.23.0$, licence BSD-3-Clause.

4.3.3 matplotlib / seaborn

Matplotlib [9] est utilisé pour générer les neuf graphiques du pipeline (matrices de confusion, courbes ROC, comparaisons de métriques, figures de synthèse). Seaborn [23] complète matplotlib pour les visualisations statistiques du script d'exploration (distributions, corrélations, heatmaps). Les alternatives Plotly et Bokeh offrent de l'interactivité, mais le projet nécessite uniquement des exports PNG statiques pour le rapport et les résultats sauvegardés. Versions $\geq 3.6.0$ et $\geq 0.12.0$.

4.3.4 joblib

Joblib [21] sérialise les modèles entraînés et le scaler au format `.joblib`, un format binaire optimisé pour les objets contenant de grands tableaux NumPy. Les alternatives étaient `pickle` (standard Python, mais moins performant sur les objets NumPy), ONNX (interopérable multi-langages, mais complexe à mettre en place pour des modèles scikit-learn simples) et PMML (format XML verbeux, peu utilisé en pratique). Joblib est la méthode recommandée par la documentation scikit-learn pour la persistance des modèles. Version $\geq 1.3.0$ (dépendance transitive de scikit-learn), licence BSD-3-Clause. Limite principale : format Python-only, ce qui rend les modèles non utilisables directement depuis un autre langage.

4.4 API back-end

4.4.1 Flask

Flask [17] est un micro-framework WSGI qui fournit le routage HTTP, la gestion des blueprints et les handlers d'erreur, sans imposer de couche d'abstraction base de données ni de validation intégrée. L'alternative principale était FastAPI, qui offre une validation automatique via Pydantic et le support natif de l'asynchrone. Flask a été préféré pour sa maturité, sa simplicité de configuration (un seul blueprint suffit pour les quatre routes du projet) et le fait que les prédictions scikit-learn sont des opérations synchrones rapides ne nécessitant pas d'async. Django REST Framework a été écarté car trop lourd pour une API de quatre endpoints. Version $\geq 3.0.0$, licence BSD-3-Clause. Configuration retenue : CORS ouvert (`origins="*"`), Gunicorn en production.

4.4.2 Marshmallow

Marshmallow [12] assure la validation et la désérialisation des payloads JSON entrants. Le schéma `PredictionInputSchema` déclare les types, les bornes (âge entre 18 et 100, taux d'intérêt entre 0 et 30) et les valeurs catégorielles autorisées. L'alternative Pydantic est plus moderne et offre une validation similaire, mais Marshmallow a été choisi pour sa compatibilité historique avec Flask et la distinction claire entre schémas de validation (entrée) et de sérialisation (sortie). Version $\geq 3.0.0$, licence MIT.

4.4.3 SQLAlchemy

SQLAlchemy [1] est utilisé comme ORM pour persister les prédictions. Le modèle `Prediction` déclare une table contenant les 11 features d'entrée, le modèle utilisé et les résultats (probabilité, niveau de risque, décision) pour LR et DT. L'intérêt principal de SQLAlchemy est son support multi-moteurs : en développement et en tests, l'application utilise SQLite (y compris en mémoire pour les tests), tandis qu'en production Docker, elle se connecte à PostgreSQL via la variable `DATABASE_URL`. Ce changement de backend ne nécessite aucune modification de code. Alternatives écartées : Peewee (moins de fonctionnalités), Django ORM (lié au framework Django). Version $\geq 2.0.0$, licence MIT.

4.4.4 Gunicorn

Gunicorn [7] est le serveur WSGI utilisé en production pour servir l'application Flask. Il remplace le serveur de développement intégré à Flask, qui n'est pas conçu pour gérer la concurrence. Le Dockerfile de l'API le lance avec `gunicorn --bind 0.0.0.0:8000 app.main:app`. Les alternatives étaient uWSGI (plus configurable mais plus complexe) et Waitress (portable sur Windows, non pertinent ici). Version $\geq 21.0.0$, licence MIT.

4.4.5 Flask-CORS / psycopg2-binary

Flask-CORS [5] active le partage de ressources entre origines (CORS) pour permettre au front-end, servi sur le port 5173, d'appeler l'API sur le port 8000. La configuration actuelle autorise toutes les origines (`origins="*"`), ce qui est acceptable en développement mais devrait être restreint en production. Psycopg2-binary [2] est le driver PostgreSQL utilisé par SQLAlchemy pour communiquer avec la base de données en production Docker. La variante `-binary` inclut les bibliothèques C pré-compilées, évitant la nécessité de compiler `libpq` dans le conteneur.

4.5 Front-end

4.5.1 Vue 3

Vue 3 [25] est le framework JavaScript retenu pour l'interface utilisateur. Il fournit un système de composants réactifs, une Composition API concise et des Single File Components (`.vue`) qui regroupent template, script et style dans un même fichier. Les alternatives considérées étaient React (écosystème plus vaste mais JSX moins lisible pour un projet de cette taille), Svelte (compilateur innovant mais communauté plus restreinte) et Angular (framework complet mais lourd pour quatre vues). Vue 3 a été retenu pour sa courbe d'apprentissage modérée et sa légèreté. L'application n'utilise pas de state manager (Pinia) car l'état est local à chaque vue. Version $\geq 3.5.25$, licence MIT.

4.5.2 Vite

Vite [26] est le bundler et serveur de développement du front-end. Il remplace Webpack en proposant un démarrage quasi instantané grâce au *native ESM* et un rechargement à chaud (HMR) très rapide. La configuration est minimale : le fichier `vite.config.js` déclare les plugins Vue et Tailwind, et ouvre le serveur sur `0.0.0.0` pour être accessible depuis l'hôte Docker. Les alternatives Webpack (configuration complexe) et Parcel (moins de contrôle sur les plugins) ont été écartées. Version $\geq 7.3.1$, licence MIT.

4.5.3 Tailwind CSS 4

Tailwind CSS [24] est un framework CSS *utility-first* qui permet de styler les composants directement dans le HTML via des classes utilitaires (`bg-white`, `rounded-lg`, `px-4`). Cette approche élimine le besoin d'écrire des fichiers CSS dédiés et garantit la cohérence visuelle. Les alternatives Bootstrap (composants prédéfinis, moins flexible) et CSS vanilla (effort de maintenance supérieur) ont été écartées. L'intégration avec Vite se fait via le plugin `@tailwindcss/vite` déclaré dans la configuration. Version $\geq 4.1.18$, licence MIT.

4.5.4 Vue Router 4

Vue Router [22] gère la navigation côté client entre les trois routes de l'application : / (formulaire de prédiction), /history (liste des prédictions) et /history/:id (détail). Le routeur utilise le mode `createWebHistory` pour des URL propres sans hash. Aucune garde d'authentification n'est configurée. Version $\geq 4.6.4$, licence MIT.

4.6 Infrastructure

4.6.1 Docker / Docker Compose

Docker [3] et Docker Compose [4] assurent la conteneurisation de l'application. Le projet utilise trois Dockerfiles distincts : un pour le pipeline ML (`python:3.11-slim` avec Make installé), un pour l'API (`python:3.11-slim` avec Gunicorn) et un pour le front-end (`node:20-alpine` avec Vite). Docker Compose orchestre les trois services et gère le réseau inter-conteneurs. L'alternative Podman est compatible mais moins répandue en environnement académique. Vagrant a été écarté car il virtualise un OS complet, ce qui est plus lourd que la conteneurisation. Limites : le surcoût mémoire des conteneurs et la complexité du réseau Docker (résolution DNS entre services). Licence Apache 2.0.

4.6.2 PostgreSQL 16

PostgreSQL [19] est la base de données relationnelle utilisée en production pour persister les prédictions. L'image Docker `postgres:16-alpine` est légère et démarre rapidement. La base `credit_risk` est créée automatiquement au premier démarrage via les variables d'environnement du Compose. Les alternatives considérées étaient MySQL (moins de fonctionnalités SQL avancées), MongoDB (non pertinent pour des données tabulaires structurées) et SQLite en production (pas de gestion de concurrence). PostgreSQL offre la robustesse, le support des transactions ACID et une intégration native avec SQLAlchemy via psycopg2. Licence PostgreSQL (permissive, type MIT).

4.6.3 Make

GNU Make [18] orchestre le pipeline ML via un fichier `Makefile` déclarant dix cibles (voir Tableau 5.1). L'intérêt principal de Make est la gestion déclarative des dépendances entre cibles : la cible `train` dépend de `split`, et `plots` dépend de `train`. Ainsi, `$ make plots` exécute automatiquement les étapes préalables si nécessaire. Les alternatives étaient des scripts bash (pas de gestion de dépendances), Invoke (Python, moins standard), Snakemake (orienté bioinformatique) et DVC (versionnement de données, surdimensionné ici). Make a été retenu pour son ubiquité sur les systèmes Unix et sa syntaxe minimaliste. Licence GPL.

4.7 Gestion de versions

4.7.1 Git

Git [20] assure le versionnement du code du projet. C'est l'outil libre emblématique du développement logiciel : il trace chaque modification (auteur, contenu, date), permet de revenir à un état antérieur en cas de régression, et structure le travail en commits atomiques dont les messages documentent l'évolution du projet. Son modèle décentralisé permet de travailler hors ligne et fait de chaque clone une sauvegarde complète de l'historique. Les alternatives considérées étaient Subversion (SVN, centralisé : un serveur unique constitue un point de défaillance et le travail hors ligne est limité) et Mercurial (décentralisé et ergonomique, mais communauté et écosystème plus restreints). Git a été retenu pour son statut de standard de fait, son intégration native avec les forges logicielles (GitHub, GitLab) et la richesse de son outillage. Licence : GPLv2, une licence libre *copyleft* - les redistributions de Git lui-même doivent rester sous GPL, ce qui ne concerne en rien le code versionné avec l'outil. Limite assumée : l'historique du projet est encore réduit, le versionnement ayant été mis en place tardivement ; des commits plus réguliers et plus fins au fil du développement auraient mieux documenté la démarche, et c'est la pratique adoptée pour les corrections récentes du projet.

4.8 Tests et qualité

4.8.1 pytest

Pytest [11] est le framework de tests du projet, configuré via `pytest.ini` avec les options `-v --tb=short` pour un affichage détaillé et des tracebacks concis. Pytest a été préféré au module standard `unittest` pour sa syntaxe plus concise (simples fonctions `assert` au lieu de méthodes de classe), son système de fixtures (utilisé pour le client Flask de test) et sa découverte automatique des fichiers `test_*.py`. L'alternative `nose2` est en déclin. Version $\geq 7.0.0$, licence MIT.

4.8.2 SQLite in-memory

L'isolation des tests est assurée par une base SQLite en mémoire. Le fichier `conftest.py` définit `DATABASE_URL=sqlite:///memory:` avant tout import de l'application, ce qui force SQLAlchemy à utiliser une base éphémère créée à chaque session de tests. Cette approche élimine la dépendance à PostgreSQL pour l'exécution des tests et garantit que chaque lancement part d'un état vierge. Le flag `check_same_thread=False` est activé dans la configuration de l'engine pour permettre l'utilisation de SQLite dans un contexte multi-thread Flask.

4.8.3 pytest-cov

Pytest-cov [16] mesure la couverture de code des tests, c'est-à-dire la proportion de lignes du module `app/` effectivement exécutées par la suite de tests. La cible `$ make coverage` produit un rapport ligne à ligne dans le terminal (option `--cov-report=term-missing`), qui identifie les branches non testées. La mesure actuelle s'établit à 97% sur le module de l'API ; elle ne couvre ni les scripts ML ni le front-end (voir chapitre 6). L'alternative consistant à utiliser `coverage.py` seul a été écartée au profit du plugin `pytest`, qui s'intègre à la suite existante sans configuration supplémentaire. Licence MIT.

4.8.4 GitHub Actions (intégration continue)

GitHub Actions [6] exécute automatiquement la suite de tests à chaque `push` et `pull request`, via le workflow `.github/workflows/tests.yml` : installation de Python 3.11, installation des dépendances, puis `$ make test` et `$ make coverage`. Le dataset et les scripts étant versionnés, la CI réentraîne les modèles avant de lancer les tests, exactement comme en local - la chaîne complète est donc validée à chaque modification. Les alternatives étaient GitLab CI (équivalent, mais le dépôt est hébergé sur GitHub) et Jenkins (auto-hébergé, surdimensionné pour un projet individuel). GitHub Actions a été retenu pour son intégration directe à la forge et sa gratuité pour les dépôts publics. Limite : le service lui-même n'est pas un logiciel libre, seuls les *runners* sont open source (licence MIT) ; c'est un compromis assumé, GitLab CI constituant l'alternative libre auto-hébergeable.

4.9 Tableau récapitulatif

Tableau 4.1 - Tableau récapitulatif des outils

Outil	Rôle	Vers.	Justification	Altern.	Lic.
Python	Scripts, API	3.11	Écosyst. ML	R, Julia	PSF
JavaScript	Front-end	ES2022	Standard web	TypeScript	-
scikit-learn	ML (LR, DT)	≥1.2	API cohérente	XGBoost	BSD-3
pandas	DataFrames	≥1.5	Standard sklearn	Polars	BSD-3
numpy	Calculs num.	≥1.23	Dépend. fond.	-	BSD-3
matplotlib	Graphiques	≥3.6	Export PNG	Plotly	PSF
seaborn	Visu. stats	≥0.12	Heatmaps	Plotly	BSD-3
joblib	Sérialisation	≥1.3	Optimisé numpy	pickle	BSD-3
Flask	API REST	≥3.0	Léger, pas d'async	FastAPI	BSD-3
Marshmallow	Validation	≥3.0	Schémas décl.	Pydantic	MIT
SQLAlchemy	ORM	≥2.0	Multi-DB	Peewee	MIT
Gunicorn	Serveur WSGI	≥21	Standard prod	uWSGI	MIT

Suite page suivante

Outil	Rôle	Vers.	Justification	Altern.	Lic.
Flask-CORS	CORS	≥4.0	Intégr. Flask	-	MIT
psycpg2	Driver PG	≥2.9	Standard PG	asyncpg	LGPL
Vue 3	Framework SPA	≥3.5	Composition API	React	MIT
Vite	Bundler	≥7.3	HMR rapide	Webpack	MIT
Tailwind	CSS utilitaire	≥4.1	Responsive	Bootstrap	MIT
Vue Router	Routing	≥4.6	Standard Vue	-	MIT
Docker	Conteneurs	-	Reproductibilité	Podman	Apache
PostgreSQL	BDD	16	SQL standard	MySQL	PG Lic.
Make	Pipeline	-	Ubiquité Unix	Invoke	GPL
Git	Versionnement	-	Standard de fait	Mercurial	GPLv2
pytest	Tests	≥7.0	Fixtures, assertions	unittest	MIT
pytest-cov	Couverture	≥4.0	Intégr. pytest	coverage.py	MIT
GitHub Actions	CI	-	Intégr. forge	GitLab CI	-

4.10 Licence du projet

L'analyse des licences des dépendances pose naturellement la question de la licence du projet lui-même. Le code est publié sous licence MIT [14] (fichier **LICENSE** à la racine du dépôt) : une licence libre permissive, courte et largement comprise, qui autorise la réutilisation, la modification et la redistribution du code - y compris dans un cadre propriétaire - sous seule condition de conserver la notice de copyright. Ce choix est adapté à un projet académique destiné à être lu, réutilisé et prolongé.

Il est juridiquement compatible avec l'ensemble des dépendances : les bibliothèques intégrées à l'application (Flask, scikit-learn, Vue, etc.) sont sous licences permissives (BSD, MIT, Apache 2.0, PSF), qui n'imposent aucune contrainte de licence au code qui les utilise. Les licences copyleft présentes dans le projet ne s'appliquent pas davantage au code : la GPL de Make et la GPLv2 de Git concernent les outils eux-mêmes (utilisés respectivement comme orchestrateur de build et comme gestionnaire de versions, sans lien de code avec l'application), et la LGPL de psycpg2 autorise expressément l'utilisation par liaison dynamique sans étendre ses obligations au programme appelant. Utiliser des outils libres n'impose donc pas leur licence au projet - c'est précisément la distinction entre licences permissives et licences copyleft que ce choix illustre.

Chapitre 5

Environnement et reproductibilité

5.1 Installation locale

L'installation locale nécessite Python 3.8+ (3.11 recommandé), pip et Node.js 20+. Après clonage du dépôt, un environnement virtuel Python est créé pour isoler les dépendances. Deux fichiers `requirements.txt` sont installés : celui du répertoire `back-end/` pour les scripts ML (pandas, numpy, matplotlib, seaborn, scikit-learn) et celui de `back-end/app/` pour l'API (Flask, SQLAlchemy, Marshmallow, Gunicorn, etc.). Le front-end est installé séparément via `npm install`. Le pipeline complet est ensuite exécuté par `$ make all`. Les Tableaux 5.2, 5.3 et 5.4 détaillent les dépendances et leurs versions minimales.

```
1 # Cloner et préparer l'environnement
2 git clone <url-du-repo>
3 cd PROGRAMME
4 python3 -m venv .venv
5 source .venv/bin/activate
6 pip install -r back-end/requirements.txt
7 pip install -r back-end/app/requirements.txt
8
9 # Pipeline ML complet
10 make all          # split -> train -> plots
11
12 # Tests
13 make test         # 25 tests pytest
14
15 # Front-end (développement local)
16 cd front-end
17 npm install
18 npm run dev       # http://localhost:5173
```

Listing 5.1 - Installation et exécution du pipeline

5.2 Déploiement Docker

Le déploiement Docker requiert au préalable l'exécution de `$ make train` pour générer les modèles `.joblib`, qui sont copiés dans l'image de l'API au moment du build ; la cible `$ make docker-up` enchaîne automatiquement ces deux étapes dans le bon ordre. Si les modèles manquent malgré tout, l'API refuse de démarrer avec un message explicite indiquant la commande à lancer, plutôt qu'une erreur confuse. La commande `docker compose up -d --build` construit les trois images et démarre les conteneurs dans l'ordre garanti par les healthchecks (PostgreSQL prêt avant l'API, API saine avant le front-end). Le service PostgreSQL est accessible sur le port 5432, l'API sur le port 8000 et l'interface web sur le port 5173. La vérification se fait via `curl http://localhost:8000/health`, qui retourne `{"status": "ok"}` si l'API est opérationnelle.

```
1 # Prerequis : make train (modeles .joblib requis)
2 make train
3
4 # Lancer les 3 services
5 docker compose up -d --build
6
7 # Verifier
8 curl http://localhost:8000/health      # {"status": "ok"}
9 # Interface web : http://localhost:5173
10
11 # Arrêter
12 docker compose down
```

Listing 5.2 - Déploiement avec Docker Compose

5.3 Exécution du pipeline

Le Tableau 5.1 récapitule les douze cibles du Makefile. La chaîne principale `all` enchaîne `split`, `train` et `plots`. Les cibles `explore` et `outliers` sont indépendantes et peuvent être exécutées à tout moment pour analyser les données brutes. La cible `test` dépend de `train` car les tests d'intégration de l'API nécessitent les modèles sérialisés pour effectuer des prédictions. Enfin, `clean` supprime l'ensemble des fichiers générés (résultats, modèles, données traitées), permettant de repartir d'un état vierge.

Tableau 5.1 - Cibles du Makefile

Commande	Dépendance	Description
\$ make all	split, train, plots	Pipeline complet
\$ make split	-	Split stratifié 80/20 du dataset
\$ make train	split	Entraînement LR + DT, sauvegarde .joblib
\$ make train-force	split	Réentraînement forcé (ignore le cache)
\$ make plots	train	Génération des 9 graphiques
\$ make explore	-	Exploration statistique des données
\$ make outliers	-	Détection des outliers (IQR + Z-score)
\$ make test	train	25 tests pytest
\$ make coverage	train	Tests avec couverture (pytest-cov)
\$ make docker-up	train	Entraînement puis docker compose up
\$ make clean	-	Suppression des fichiers générés
\$ make help	-	Affiche l'aide

5.4 Tableau des versions

Les tableaux suivants listent les dépendances avec leurs versions minimales requises, telles que déclarées dans les fichiers `requirements.txt` et `package.json`. Les versions exactes installées lors du développement sont documentées dans le fichier `docs/Rapport final/versions_exactes.md` (obtenues via `pip freeze` et `npm list`).

Tableau 5.2 - Dépendances Python (scripts ML)

Package	Version min.	Rôle
pandas	$\geq 1.5.0$	Manipulation de données
numpy	$\geq 1.23.0$	Calculs numériques
matplotlib	$\geq 3.6.0$	Graphiques
seaborn	$\geq 0.12.0$	Visualisation statistique
scikit-learn	$\geq 1.2.0$	ML (entraînement, métriques)

Tableau 5.3 - Dépendances Python (API Flask)

Package	Version min.	Rôle
flask	$\geq 3.0.0$	Micro-framework HTTP
flask-cors	$\geq 4.0.0$	Gestion CORS
gunicorn	$\geq 21.0.0$	Serveur WSGI production
marshmallow	$\geq 3.0.0$	Validation de schémas
sqlalchemy	$\geq 2.0.0$	ORM (PostgreSQL / SQLite)
psycopg2-binary	$\geq 2.9.0$	Driver PostgreSQL
scikit-learn	$\geq 1.2.0$	Chargement des modèles
joblib	$\geq 1.3.0$	Désérialisation .joblib
numpy	$\geq 1.23.0$	Tableaux numériques
pandas	$\geq 1.5.0$	DataFrames (predict)
pytest	$\geq 7.0.0$	Tests
pytest-cov	$\geq 4.0.0$	Couverture de code

Tableau 5.4 - Dépendances Node.js (front-end)

Package	Version	Rôle
vue	$\geq 3.5.25$	Framework SPA
vue-router	$\geq 4.6.4$	Routing client
tailwindcss	$\geq 4.1.18$	CSS utilitaire
@tailwindcss/vite	$\geq 4.1.18$	Plugin Vite pour Tailwind
vite (dev)	$\geq 7.3.1$	Bundler / dev server
@vitejs/plugin-vue (dev)	$\geq 6.0.2$	Plugin Vue pour Vite

Chapitre 6

Tests

6.1 Stratégie de tests

La suite de tests comprend 25 cas répartis dans cinq fichiers, comme le détaille le Tableau 6.1. La stratégie couvre trois niveaux : les tests unitaires vérifient les fonctions isolées (encodage des features dans `test_preprocess.py`, seuils de classification dans `test_predict.py`, validation des schémas Marshmallow dans `test_schemas.py`, absence de fuite de données dans l'imputation du pipeline dans `test_imputation.py`), tandis que les tests d'intégration dans `test_api.py` exercent les endpoints REST de bout en bout, de la requête HTTP à la réponse JSON en passant par la prédiction et la persistance en base. Ce découpage permet d'identifier rapidement l'origine d'une régression : un échec dans `test_preprocess.py` pointe vers un problème d'encodage, tandis qu'un échec dans `test_api.py` peut impliquer n'importe quelle couche de la pile.

Tableau 6.1 - Répartition des tests

Fichier	Tests	Couverture
<code>test_api.py</code>	7	Endpoints REST (health, predict, predictions, erreurs, bornes)
<code>test_imputation.py</code>	3	Anti-fuite : statistiques d'imputation issues du train uniquement
<code>test_predict.py</code>	5	Seuils de classification <code>_classify</code> (Safe, Moyen, Risque, limites)
<code>test_preprocess.py</code>	3	Encodage features (shape, mappings catégoriels, ordre)
<code>test_schemas.py</code>	7	Validation Marshmallow (payload, bornes dont <code>loan_amnt</code> , catégoriel, requis)
Total	25	

6.2 Isolation

L'isolation des tests repose sur deux mécanismes. D'une part, le fichier `conftest.py` redéfinit la variable d'environnement `DATABASE_URL` vers `sqlite:///memory:` avant l'import de l'application Flask, ce qui force SQLAlchemy à créer une base éphémère en mémoire. D'autre part, la fixture `client` active le mode `TESTING` de Flask et fournit un client

HTTP de test qui simule les requêtes sans ouvrir de socket réseau. Cette approche garantit que les tests s'exécutent sans dépendance externe (pas de PostgreSQL, pas de serveur HTTP) et que chaque session de tests démarre avec une base vide.

6.3 Exécution

L'exécution des tests se fait via `$ make test`, qui dépend de la cible `train` pour s'assurer que les modèles `.joblib` existent. Le Makefile sélectionne l'interpréteur du virtualenv (`.venv/bin/python`) s'il existe, et `python3` du système sinon. Lors de la dernière exécution, les 25 tests passent avec succès en moins d'une seconde, confirmant l'absence de régression sur la validation des entrées, l'encodage des features, l'imputation, les seuils de classification et les quatre endpoints de l'API.

6.4 Intégration continue et couverture

Une CI GitHub Actions (`.github/workflows/tests.yml`) rejoue l'ensemble de la chaîne à chaque `push` et `pull request` : installation des dépendances, entraînement des modèles (le dataset étant versionné), exécution des 25 tests puis mesure de la couverture. Toute régression est ainsi détectée avant intégration, sans dépendre d'une exécution manuelle.

La couverture de code, mesurée par `pytest-cov` via `$ make coverage`, s'établit à 97 % des 193 instructions du module `app/` (les cinq lignes non couvertes correspondent au handler d'erreur 500 et au message d'absence des modèles). Cette mesure doit être interprétée avec prudence : elle porte uniquement sur le code de l'API - les scripts du pipeline ML ne sont couverts qu'indirectement par `test_imputation.py` et le front-end Vue ne dispose d'aucun test automatisé - et une ligne exécutée n'est pas pour autant une ligne correctement vérifiée. Elle fournit néanmoins un indicateur objectif pour repérer les zones non testées.

Chapitre 7

Limites et améliorations

7.1 Limites actuelles

Plusieurs limites identifiées découlent des choix de simplification évoqués en introduction. Sur le plan de la sécurité, l'API est publique (pas d'authentification), la politique CORS autorise toutes les origines (`origins="*"`), et les identifiants de la base PostgreSQL sont écrits en clair dans `docker-compose.yml`. Sur le plan méthodologique, la profondeur de l'arbre est désormais choisie par validation croisée à 5 plis, mais l'évaluation finale repose toujours sur un unique split 80/20 : des intervalles de confiance (validation croisée imbriquée ou bootstrap) rendraient les métriques moins sensibles à la partition aléatoire. Le déséquilibre des classes (78/22 %) a été quantifié à travers les variantes `class_weight="balanced"` (Tableau 3.5) ; le modèle déployé reste néanmoins la version non pondérée, et les techniques de rééchantillonnage comme SMOTE n'ont pas été explorées. Sur le plan opérationnel, le projet dispose désormais d'une CI et d'une mesure de couverture, mais toujours pas de monitoring ni de logging structuré, et le conteneur front-end utilise le serveur de développement Vite au lieu d'un build de production servi par un serveur statique. Enfin, le schéma de la base de données n'est pas versionné (pas d'Alembic), ce qui compliquerait l'évolution du modèle `Prediction`.

7.2 Améliorations possibles

Plusieurs axes d'amélioration concrets se dégagent. La CI GitHub Actions existante pourrait être étendue au linting, à la construction des images Docker et à la publication des rapports de couverture. L'introduction d'une authentification JWT sécuriserait l'accès à l'API. Côté ML, des techniques de rééchantillonnage (SMOTE) restent à explorer en complément des variantes pondérées déjà évaluées, et une recherche d'hyperparamètres plus large (`GridSearchCV` sur l'ensemble des hyperparamètres, au-delà de la seule profondeur de l'arbre) permettrait d'optimiser les performances sans intervention manuelle. Sur l'infrastructure, un reverse proxy nginx devant Gunicorn améliorerait la gestion des connexions et permettrait de servir le front-end compilé (`npm run build`) en tant que fichiers statiques au lieu d'utiliser le serveur de développement Vite. L'introduction d'Alembic pour les migrations de schéma et de secrets Docker pour les mots de passe compléterait la sécurisation du déploiement. Enfin, un système de monitoring (Prometheus + Grafana) et de logging structuré (JSON) rendrait l'application observable en production.

Chapitre 8

Conclusion

Ce projet a permis de construire un système complet de prédiction du risque de crédit, depuis l'analyse exploratoire des données jusqu'au déploiement conteneurisé de l'application. Les cinq objectifs initiaux ont été atteints : le pipeline ML est reproductible via un Makefile qui enchaîne split, entraînement et visualisation ; l'API REST expose les prédictions des deux modèles avec validation des entrées et persistance en base ; l'interface Vue 3 offre un formulaire de saisie, l'affichage des résultats et un historique consultable ; Docker Compose orchestre les trois services (PostgreSQL, API, front-end) en une seule commande ; et 25 tests automatisés, exécutés à chaque push par une CI GitHub Actions, couvrent les couches critiques du système avec 97 % de couverture sur le module de l'API.

L'ensemble du projet repose exclusivement sur des outils libres, conformément à l'objectif du cours. Le Tableau 4.1 recense les 25 outils utilisés, tous distribués sous des licences libres : des licences permissives (BSD, MIT, Apache, PSF) pour les bibliothèques intégrées à l'application, et des licences copyleft (GPL pour Make et Git, LGPL pour psycopg2) pour les outils de développement - le copyleft impose que les redistributions de l'outil restent sous la même licence, mais ne s'applique pas au code produit avec ces outils. Le projet lui-même est publié sous licence MIT [14]. Ce choix garantit la transparence, la possibilité de modification et l'absence de coûts de licence.

La reproductibilité est assurée par trois mécanismes complémentaires : le Makefile déclare les dépendances entre étapes du pipeline, les fichiers `requirements.txt` et `package.json` fixent les versions minimales des dépendances, et les Dockerfiles figent l'environnement d'exécution (Python 3.11, Node 20, PostgreSQL 16). Un tiers peut ainsi reconstruire l'ensemble du système à partir du seul dépôt de code.

Les limites identifiées (absence d'authentification, de CI/CD, de traitement du déséquilibre des classes) constituent autant de pistes d'amélioration pour une mise en production réelle. Ce projet démontre néanmoins qu'il est possible, avec des outils libres matures et bien documentés, de mettre en place un pipeline ML de bout en bout intégrant entraînement, serving et interface utilisateur dans un cadre reproductible et conteneurisé.

Bibliographie

- [1] Mike Bayer. SQLAlchemy - the database toolkit for Python. <https://www.sqlalchemy.org/>, 2006. Version 2.0+, licence MIT. Consulté en juin 2026.
- [2] Federico Di Gregorio and Daniele Varrazzo. psycopg2 - PostgreSQL adapter for Python. <https://www.psycopg.org/>, 2001. Licence LGPL. Consulté en juin 2026.
- [3] Docker, Inc. Docker - build, ship, and run any app, anywhere. <https://www.docker.com/>, 2013. Licence Apache 2.0. Consulté en juin 2026.
- [4] Docker, Inc. Docker compose - define and run multi-container applications. <https://docs.docker.com/compose/>, 2014. Licence Apache 2.0. Consulté en juin 2026.
- [5] Cory Dolphin. Flask-cors - cross-origin resource sharing for flask. <https://flask-cors.corydolphin.com/>, 2013. Licence MIT. Consulté en juin 2026.
- [6] GitHub, Inc. GitHub actions documentation. <https://docs.github.com/en/actions>, 2019. Service d'intégration continue. Consulté en juin 2026.
- [7] Gunicorn contributors. Gunicorn - Python WSGI HTTP server for UNIX. <https://gunicorn.org/>, 2010. Version 21.0+, licence MIT. Consulté en juin 2026.
- [8] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, et al. Array programming with NumPy. *Nature*, 585(7825) :357-362, 2020. doi : 10.1038/s41586-020-2649-2.
- [9] John D. Hunter. Matplotlib : A 2d graphics environment. *Computing in Science & Engineering*, 9(3) :90-95, 2007. doi : 10.1109/MCSE.2007.55.
- [10] Kaggle contributors. Credit risk dataset. <https://www.kaggle.com/datasets/laotse/credit-risk-dataset>, 2022. 32 581 lignes, 12 colonnes, licence CC0. Consulté en juin 2026.
- [11] Holger Krekel and pytest contributors. pytest : helps you write better programs. <https://pytest.org/>, 2004. Version 7.0+, licence MIT. Consulté en juin 2026.
- [12] Steven Loria. marshmallow : simplified object serialization. <https://marshmallow.readthedocs.io/>, 2014. Version 3.0+, licence MIT. Consulté en juin 2026.
- [13] Wes McKinney. Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, pages 56-61, 2010. doi : 10.25080/Majora-92bf1922-00a.

- [14] Open Source Initiative. The MIT license. <https://opensource.org/license/mit/>, 1988. Texte officiel de la licence. Consulté en juin 2026.
- [15] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn : Machine learning in Python. *Journal of Machine Learning Research*, 12 :2825-2830, 2011.
- [16] pytest-cov contributors. pytest-cov - coverage plugin for pytest. <https://pytest-cov.readthedocs.io/>, 2010. Version 4.0+, licence MIT. Consulté en juin 2026.
- [17] Armin Ronacher. Flask : A python micro web framework. <https://flask.palletsprojects.com/>, 2010. Version 3.0+, licence BSD-3-Clause. Consulté en juin 2026.
- [18] Richard M. Stallman, Roland McGrath, and Paul D. Smith. GNU make - a program for directing recompilation. <https://www.gnu.org/software/make/>, 1988. Licence GPL. Consulté en juin 2026.
- [19] The PostgreSQL Global Development Group. PostgreSQL : The world's most advanced open source relational database. <https://www.postgresql.org/>, 1996. Version 16, PostgreSQL License (MIT-like). Consulté en juin 2026.
- [20] Linus Torvalds, Junio C. Hamano, and Git contributors. Git - distributed version control system. <https://git-scm.com/>, 2005. Licence GPLv2 (copyleft). Consulté en juin 2026.
- [21] Gaël Varoquaux. joblib - lightweight pipelining with Python functions. <https://joblib.readthedocs.io/>, 2008. Licence BSD-3-Clause. Consulté en juin 2026.
- [22] Vue.js contributors. Vue router - the official router for vue.js. <https://router.vuejs.org/>, 2014. Version 4.6+, licence MIT. Consulté en juin 2026.
- [23] Michael L. Waskom. seaborn : statistical data visualization. *Journal of Open Source Software*, 6(60) :3021, 2021. doi : 10.21105/joss.03021.
- [24] Adam Wathan and Jonathan Reinink. Tailwind CSS - a utility-first CSS framework. <https://tailwindcss.com/>, 2017. Version 4.1+, licence MIT. Consulté en juin 2026.
- [25] Evan You. Vue.js - the progressive JavaScript framework. <https://vuejs.org/>, 2014. Version 3.5+, licence MIT. Consulté en juin 2026.
- [26] Evan You. Vite - next generation frontend tooling. <https://vite.dev/>, 2020. Version 7.3+, licence MIT. Consulté en juin 2026.

Annexe A

Arborescence complète du repository

```
PROGRAMME/
+-- Makefile
+-- Dockerfile
+-- docker-compose.yml
+-- README.md
+-- LICENSE
+-- .gitignore
+-- .dockerignore
+-- .github/
|   +-- workflows/tests.yml    # CI GitHub Actions
+-- back-end/
|   +-- Dockerfile.api
|   +-- pytest.ini
|   +-- requirements.txt
|   +-- app/
|       +-- __init__.py
|       +-- main.py
|       +-- config.py
|       +-- database.py
|       +-- models.py
|       +-- schemas.py
|       +-- routes.py
|       +-- predict.py
|       +-- preprocess.py
|       +-- requirements.txt
|   +-- scripts/
|       +-- split_dataset.py
|       +-- compare_with_sklearn.py
|       +-- plot_results.py
|       +-- explore_data.py
|       +-- detect_outliers.py
|       +-- analyze_missing_values.py
|   +-- tests/
|       +-- conftest.py
|       +-- test_api.py
|       +-- test_imputation.py
```

```
| |   +-- test_predict.py
| |   +-- test_preprocess.py
| |   +-- test_schemas.py
|   +-- data/raw/      data/processed/
|   +-- models/       results/
+-- front-end/
|   +-- Dockerfile
|   +-- package.json
|   +-- vite.config.js
|   +-- index.html
|   +-- src/
|       +-- main.js
|       +-- App.vue
|       +-- style.css
|       +-- api/client.js
|       +-- router/index.js
|       +-- views/ (3 fichiers .vue)
|       +-- components/ (2 fichiers .vue)
+-- docs/
    +-- Rapport final/
    +-- presentation/
```

Annexe B

Exemple de requête/réponse API

```
1 curl -X POST http://localhost:8000/predict \  
2 -H "Content-Type: application/json" \  
3 -d '{  
4     "person_age": 30,  
5     "person_income": 50000,  
6     "person_home_ownership": "RENT",  
7     "person_emp_length": 5,  
8     "loan_intent": "PERSONAL",  
9     "loan_grade": "B",  
10    "loan_amnt": 10000,  
11    "loan_int_rate": 10,  
12    "loan_percent_income": 0.2,  
13    "cb_person_default_on_file": "N",  
14    "cb_person_cred_hist_length": 5,  
15    "model_choice": "both"  
16 }'
```

Listing B.1 - Requête POST /predict

Exemple de réponse (statut 201) :

```
1 {  
2     "id": 1,  
3     "created_at": "2026-02-14T17:35:42.123456",  
4     "model_used": "both",  
5     "lr": {  
6         "probability": 0.1842,  
7         "risk_level": "Safe",  
8         "credit_decision": "Accepte totalement"  
9     },  
10    "dt": {  
11        "probability": 0.0431,  
12        "risk_level": "Safe",  
13        "credit_decision": "Accepte totalement"  
14    }  
15 }
```

Listing B.2 - Réponse JSON POST /predict

Annexe C

Captures d'écran de l'interface web

Les captures ci-dessous illustrent l'interface web de l'application après démarrage via `docker compose up`. Elles correspondent aux vues PredictView (formulaire), ResultCard (résultats), HistoryView (historique) et DetailView (détail d'une prédiction).

The screenshot displays the 'Credit Risk Predictor' web application. At the top, there are two tabs: 'Prédiction' (active) and 'Historique'. The main content area is titled 'Informations du demandeur' and contains a form with 11 input fields arranged in a 3x3 grid. The fields are: 'Âge' (text input, value 30), 'Revenu annuel (\$)' (text input, value 50000), 'Propriété' (dropdown menu, value 'Location (RENT)'), 'Années d'emploi' (text input, value 5), 'Objet du prêt' (dropdown menu, value 'Personnel'), 'Grade du prêt' (dropdown menu, value 'B'), 'Montant du prêt (\$)' (text input, value 10000), 'Taux d'intérêt (%)' (text input, value 10), 'Ratio prêt/revenu' (text input, value 0,2), 'Antécédent de défaut' (dropdown menu, value 'Non'), 'Historique crédit (années)' (text input, value 5), and 'Modèle' (dropdown menu, value 'Les deux'). A blue 'Prédire' button is located at the bottom left of the form.

Figure C.1 - Formulaire de prédiction (PredictView) : saisie des 11 features et choix du modèle.

Credit Risk Predictor

Prédiction

Historique

Historique des prédictions

#	Date	Modèle	Montant	Proba LR	Proba DT	Décision	
29	14/02/2026 16:37:09	BOTH	\$10 000	76.3%	17.0%	Refusé / Accepté totalement	Détail
28	14/02/2026 16:37:02	BOTH	\$10 000	16.6%	5.9%	Accepté totalement / Accepté totalement	Détail
27	14/02/2026 15:56:38	BOTH	\$10 000	17.2%	5.9%	Accepté totalement / Accepté totalement	Détail
26	14/02/2026 15:55:14	BOTH	\$10 000	17.2%	5.9%	Accepté totalement / Accepté totalement	Détail
25	14/02/2026 15:35:53	BOTH	\$10 000	17.2%	5.9%	Accepté totalement / Accepté totalement	Détail
24	13/02/2026 15:49:35	BOTH	\$100 000	100.0%	100.0%	Refusé / Refusé	Détail
23	13/02/2026 15:49:33	BOTH	\$10 000	17.2%	5.9%	Accepté totalement / Accepté totalement	Détail
22	13/02/2026 15:48:25	BOTH	\$10 000	17.2%	5.9%	Accepté totalement / Accepté totalement	Détail
21	13/02/2026 15:46:22	BOTH	\$9 999	27.0%	5.9%	Accepté totalement / Accepté totalement	Détail
20	13/02/2026 15:45:28	BOTH	\$10 000	27.1%	5.9%	Accepté totalement / Accepté totalement	Détail
19	13/02/2026 15:45:15	BOTH	\$10 000	27.1%	5.9%	Accepté totalement / Accepté totalement	Détail
18	13/02/2026 15:45:12	BOTH	\$10 000	26.5%	5.9%	Accepté totalement / Accepté totalement	Détail
17	13/02/2026 15:45:06	BOTH	\$10 000	24.7%	5.9%	Accepté totalement / Accepté totalement	Détail
16	13/02/2026 15:44:59	BOTH	\$10 000	25.8%	5.9%	Accepté totalement / Accepté totalement	Détail
15	13/02/2026 15:44:54	BOTH	\$10 000	23.4%	5.9%	Accepté totalement / Accepté totalement	Détail
14	13/02/2026 15:44:34	BOTH	\$10 000	17.2%	5.9%	Accepté totalement / Accepté totalement	Détail
13	13/02/2026 15:44:31	LR	\$10 000	17.2%	-	Accepté totalement	Détail
12	13/02/2026 15:44:24	BOTH	\$10 000	17.2%	5.9%	Accepté totalement / Accepté totalement	Détail
11	13/02/2026 15:44:03	BOTH	\$10 000	17.2%	5.9%	Accepté totalement / Accepté totalement	Détail
10	13/02/2026 15:43:46	BOTH	\$10 000	17.2%	5.9%	Accepté totalement / Accepté totalement	Détail
9	13/02/2026 15:39:33	BOTH	\$10 000	17.2%	5.9%	Accepté totalement / Accepté totalement	Détail
8	13/02/2026 15:36:51	BOTH	\$10 000	17.2%	5.9%	Accepté totalement / Accepté totalement	Détail

Figure C.2 - Affichage des résultats de prédiction (ResultCard) : probabilité, niveau de risque et décision pour LR et DT.

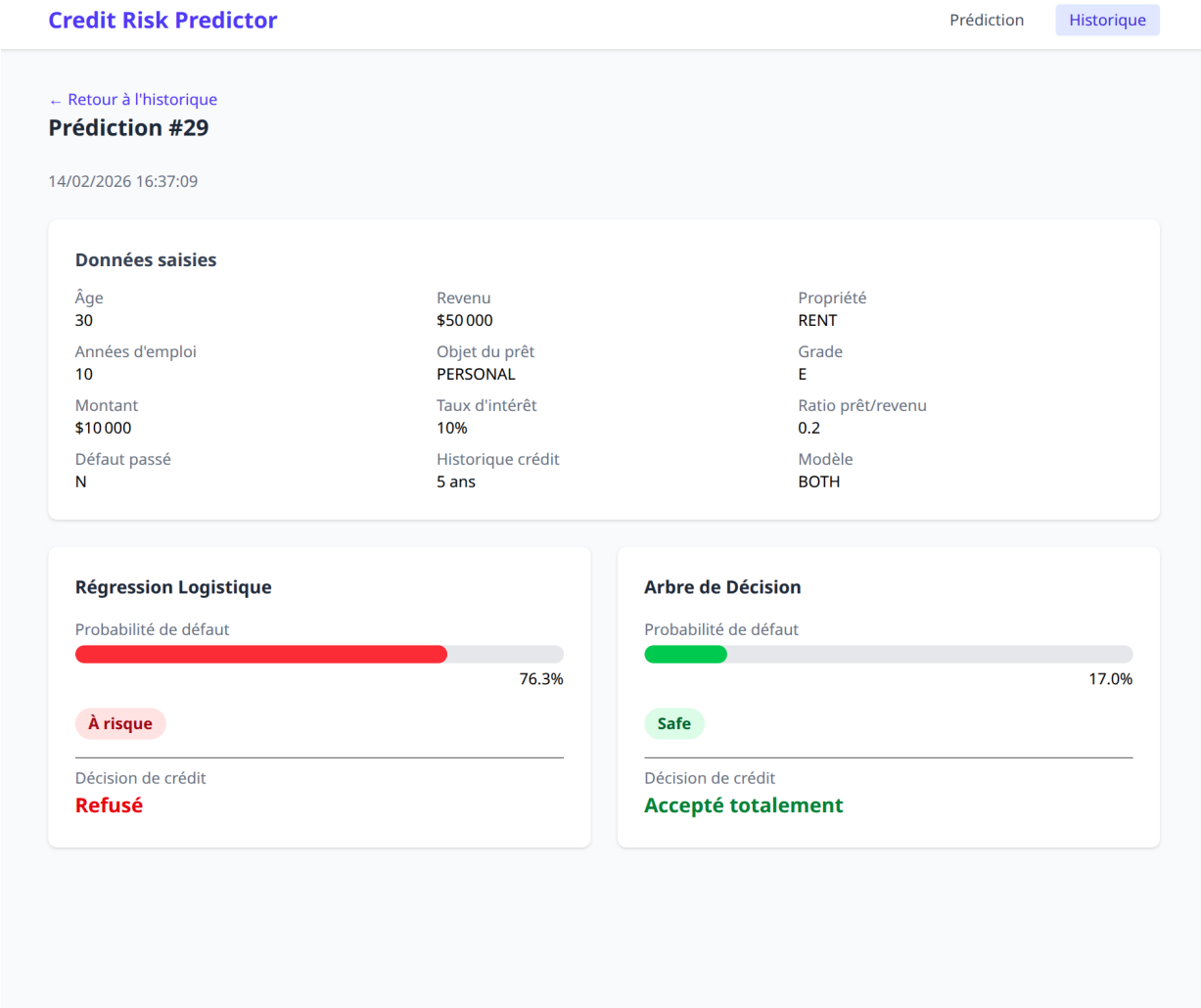


Figure C.3 - Historique des prédictions (HistoryView) : tableau paginé des prédictions enregistrées.

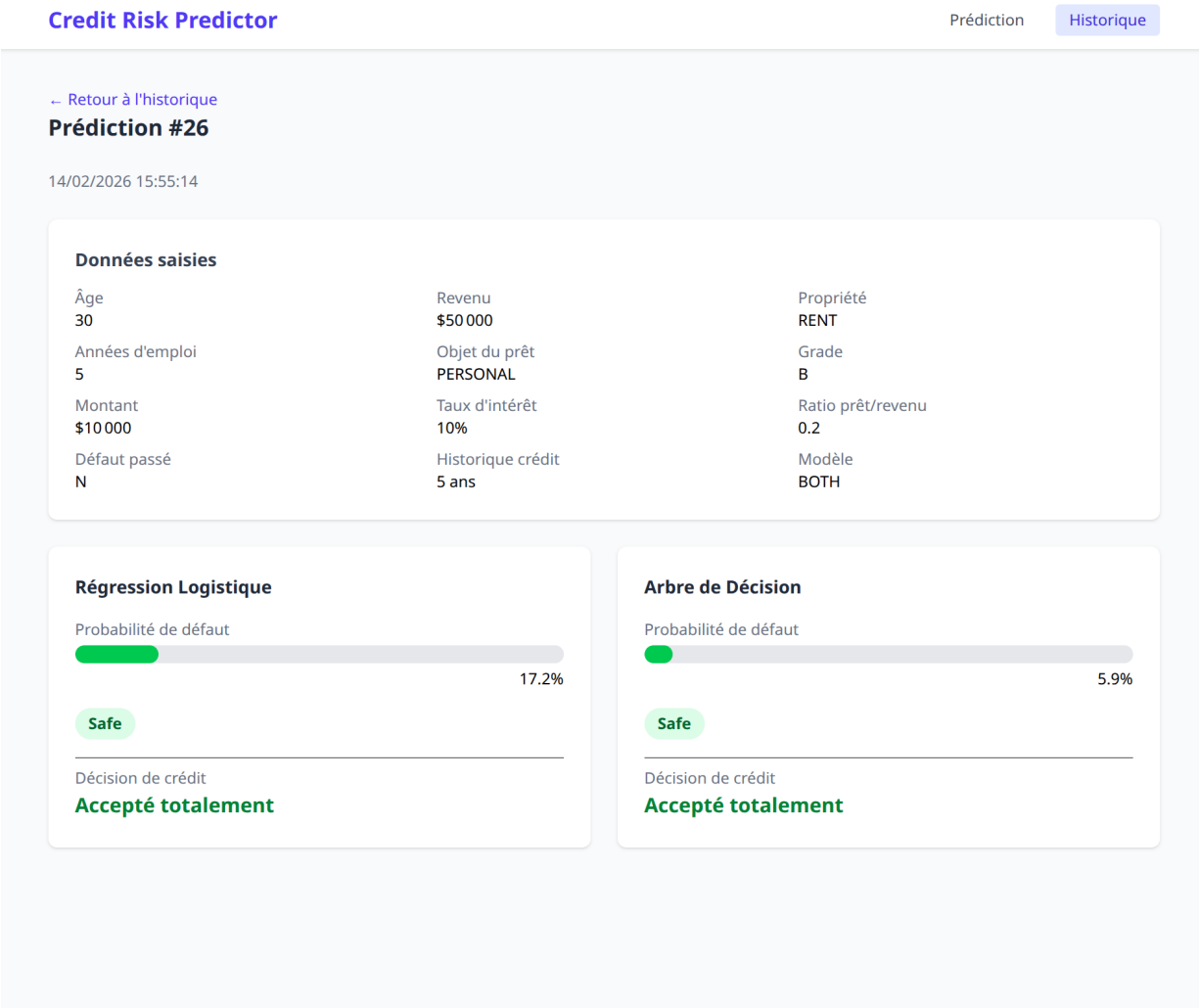


Figure C.4 - Détail d’une prédiction (DetailView) : données saisies et résultats des deux modèles.

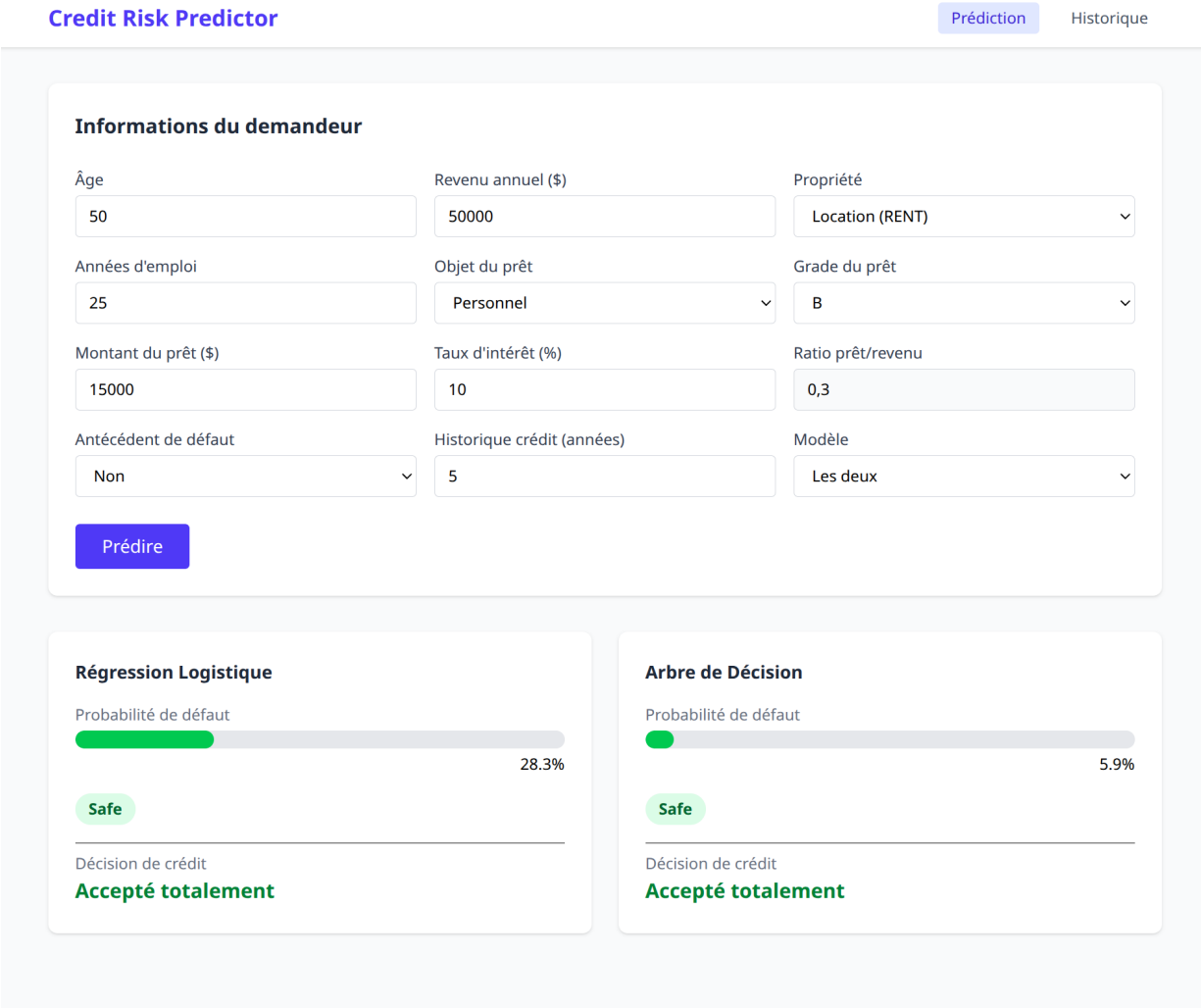


Figure C.5 - Vue complémentaire de l'interface web.